Predicting Players in Motion

NFL Big Data Bowl 2026, Prediction track • Neel Ahuja

Code on GitHub • Kaggle submission notebook

Given every player's tracking data up to the moment the ball is thrown, predict where the targeted players will be for each frame until it lands, scored by RMSE on the (x, y) coordinates. Training is the 2023 season; the hidden test is a later season, and that gap is the whole story. Lower RMSE is better; a constant-velocity baseline scores 1.645.

The model

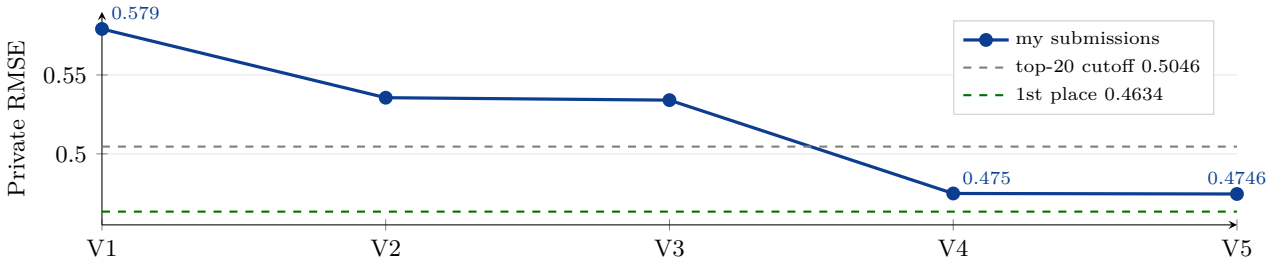
A compact CNN+Transformer, roughly 1.4M parameters to start and 4.9M at full size (train.py). Each of the 22 players becomes one token: a depthwise 1-D CNN reads that player's last 20 frames (position, speed, acceleration, heading and orientation as sine/cosine, and vectors to the ball's landing spot and the intended receiver). A pre-norm SwiGLU Transformer then lets the tokens attend across each other, since a defender's path depends on the receiver's, and a convolutional head decodes each into a per-frame displacement plus its own variance out to 48 frames. Training uses a Gaussian negative-log-likelihood (the model predicts its own error bars) with velocity and acceleration auxiliary heads, heavy geometric augmentation (random rotation, mirroring, early starts), EMA weight averaging, and a horizontal-flip test-time average.

Autoresearch: tuning as an experiment loop

Rather than hand-tune, I ran the project as a Karpathy-style autoresearch loop. prepare.py is frozen: it builds the data and defines the RMSE metric and the validation split, so nothing I optimize can quietly change how it is judged. train.py is the only file I edit, each run gets a fixed wall-clock budget so two experiments are comparable, and every result is re-scored from saved predictions and logged with its git commit to results.tsv. Twelve overnight hours produced 33 tracked experiments. The biggest lever it found: the published recipe over-weights the velocity/acceleration loss for my training budget, so cutting that weight and raising the learning rate alone moved the model from 0.94 to 0.63.

From holdout to leaderboard

My champion scored 0.478 on held-out weeks but 0.579 on the real test, because the holdout was later weeks of the same 2023 season while the test is a new one. So I pointed the loop at the leaderboard itself, treating each submission as a labeled data point, and swept the levers that move an RMSE like this: learning rate, model width and depth, the auxiliary-loss weight, augmentation, the decoder, and the ensemble makeup. Scaling to full size and 37k steps took a single model to 0.536, and fold ensembling to 0.534. The largest jump was not a hyperparameter: a parity test replayed raw plays through the submission path and the local scorer, they disagreed by 0.05, and the diff showed one feature (frames-to-predict) was zeroed for every unscored player at inference. Fixing that single line dropped the identical ensemble to 0.475.



Five submitted versions; the drop at V4 is the inference-bug fix. Lower is better.

The final model

A four-way ensemble of the full network, each 4.9M parameters (64 channels per feature, a 256-wide 3-layer Transformer) trained on a different 2023 fold. Three members integrate per-frame displacements and one uses the direct head, which decorrelates their errors; each is averaged with its own flip and its own normalization. Final private RMSE 0.47458, inside the top-20 cutoff of 0.5046 and about 0.011 off the winning 0.4634. What remains is single-model quality, not ensembling: the next steps are full-data training with no held-out fold and rotational test-time augmentation.